数字电路与逻辑设计

高英

电子工程学院

教四楼 – 126

前言

课程安排：综合课题设计实验（8周完成，题目自选，每个题目不超过7人）。

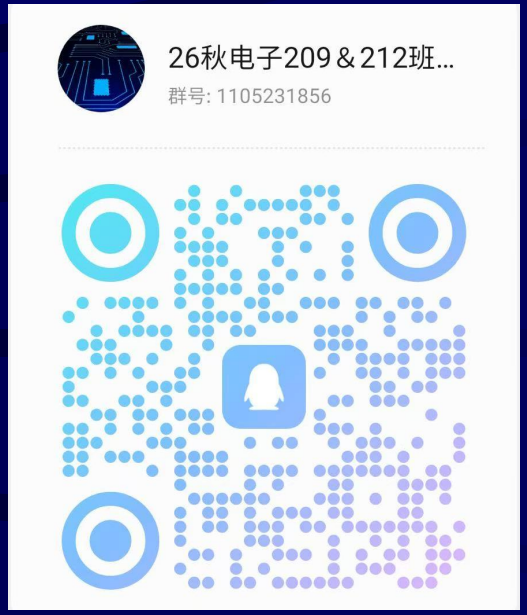
周次	内容
3	讲授内容：系统设计方法、状态机、综合题目要求等。 实验内容：学生确定选题、讨论设计方案。
4~9	开放实验：每位同学至少参加2次开放实验，根据选定题目要求完成电路的设计、仿真与硬件调试；在9月24日24:00之前和验收前提交设计方案。
9-10	验收答辩：功能验收，并根据总体框图随机选择一个模块进行代码、仿真、设计思路和实现方法的提问答辩。

成绩评定方式

- 平时20%
  - ✓ 平时实验情况（含开放）：包括考勤、纪律、实验台整理（实验结束时关闭计算机等仪表，保持实验台清洁整齐）等。
  - ✓ 设计方案：9月24日24:00之前云课堂提交初步设计方案，验收前提交实际设计方案，内容包括总体框图和主要模块的详细设计。
- 验收答辩 50%
  - ✓ 功能验收：包括实现的各项功能情况，分必做部分、选做或自拟部分。
  - ✓ 提问答辩：随机选择实际设计方案中总体框图的一个模块，进行代码、仿真波形、设计及实现方法答辩。
- 实验报告30%

实验报告要求

	内　　容	分数
1	设计课题的任务要求	5
2	系统设计（包括设计思路、总体框图、分块设计）	20
3	仿真波形及波形分析	25
4	实际实现功能说明及资源利用情况（编译报告截图）	5
5	故障及问题分析	15
6	总结和结论	10
7	AI工具使用情况（包括使用的工具、提示词）	10
8	顶层文件的HDL代码（有注释，整体工程单独提交）	10



方式一（手机访问）

2026-数字电路与逻辑设计实...

(2024211209)

请使用手机微信扫码

方式二（PC访问）

请将此段复制至微信群中：

2026-数字电路与逻辑设计实验_下的同学，欢迎您来到Easy云课堂，请直接点击链接：

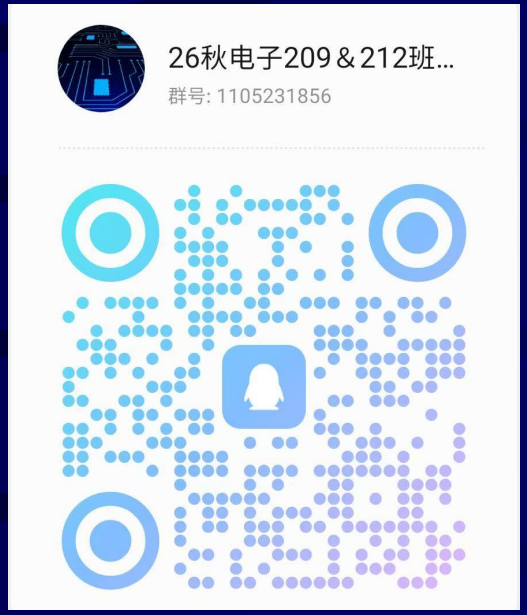
https://cloud-api.easyketang.com/admin/join_course?sign=SFpKQnpSeTc=&new_link=1

加入课程，开启不一样的学习之旅!

加入课程后的访问方式：

1. 微信小程序：Easy云课堂
2. PC端Edge或Chrome浏览器访问：https://www.easyketang.com
3. 进入云课堂后如没有看到课程，请输入课程码：HZJBzRy7。

复制信息



方式一（手机访问）

2026-数字电路与逻辑设计实...

(2024211212)

请使用手机微信扫码

方式二（PC访问）

请将此段复制至微信群中：

2026-数字电路与逻辑设计实验_下的同学，欢迎您来到Easy云课堂，请直接点击链接：

https://cloud-api.easyketang.com/admin/join_course?sign=Z1ptN0VmTWU=&new_link=1

加入课程，开启不一样的学习之旅！

加入课程后的访问方式：

1. 微信小程序：Easy云课堂
2. PC端Edge或Chrome浏览器访问：https://www.easyketang.com
3. 进入云课堂后如没有看到课程，请输入课程码：gZm7EfMe。

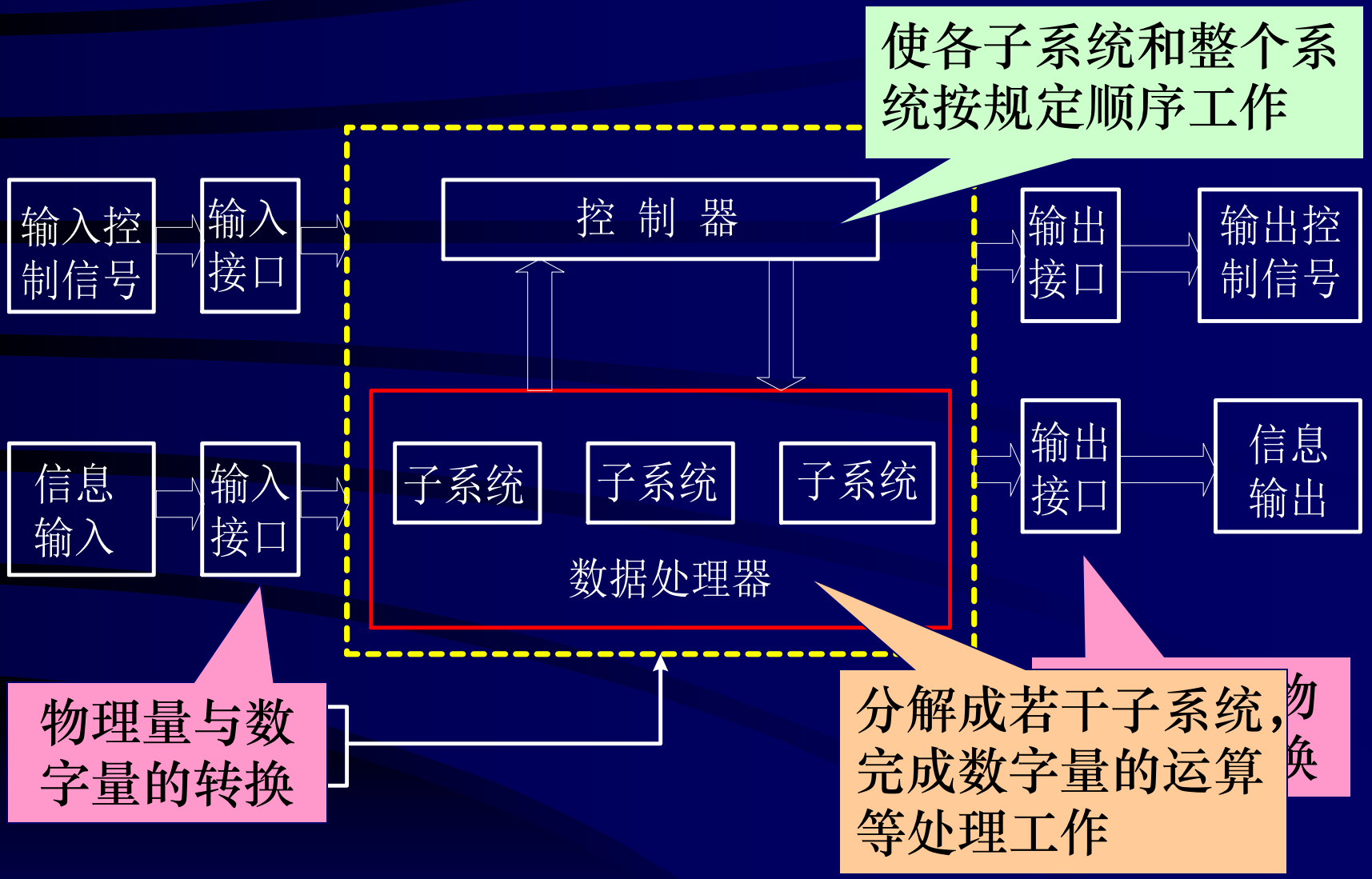
复制信息

数字系统的设计

一、数字系统概述

1、定义：数字系统是指由若干数字电路和逻辑部件构成的能够处理或传送数字信息的设备。

2、组成部分：数字系统通常可以分为三个部分：输入输出接口、数据处理器、控制器。


使各子系统和整个系统按规定顺序工作
输入控制信号
输入接口
控 制 器
输出接口
输出控制信号
信息输入
输入接口
子系统
子系统
子系统
数据处理器
输出接口
信息输出
物理量与数字量的转换
分解成若干子系统，完成数字量的运算等处理工作

数字系统结构框图

3、数字系统的特点

- 有无控制器是区别数字系统与功能部件（数字单元电路）的标志。
- 凡是有控制器，且能按照一定程序进行数据处理的系统，不论其规模大小，均称之为数字系统；否则，只能是功能部件或是数字系统中的子系统。

二、数字系统设计方法

1、传统设计法

- ✓ 纯硬件电路设计，常用试凑法；
- ✓ 所用器件为门电路、触发器和中规模集成电路；
- ✓ 所设计的电路需反复调试，比较适合小规模数字系统；
- ✓ 所用到的元器件较多，电路体积较大，出故障的几率比较高，可靠性较差。

2、现代设计法

- 硬件逻辑设计、软件逻辑设计及兼有二者优点的集成电路ASIC设计；
- 强调系统性、清晰性和可靠性；
- 将系统层层划分为较为简单部件的自顶向下的设计方法；
- 要求设计者对所设计系统要进行整体考虑，反复推敲，全面消化理解。

- 自顶向下设计步骤

（1）分析系统设计要求，明确系统逻辑功能，确定初步方案，进行系统设计和描述；

  - 描述方法（行为描述）：方框图、定时图（时序图）、逻辑流程图和硬件描述语言。

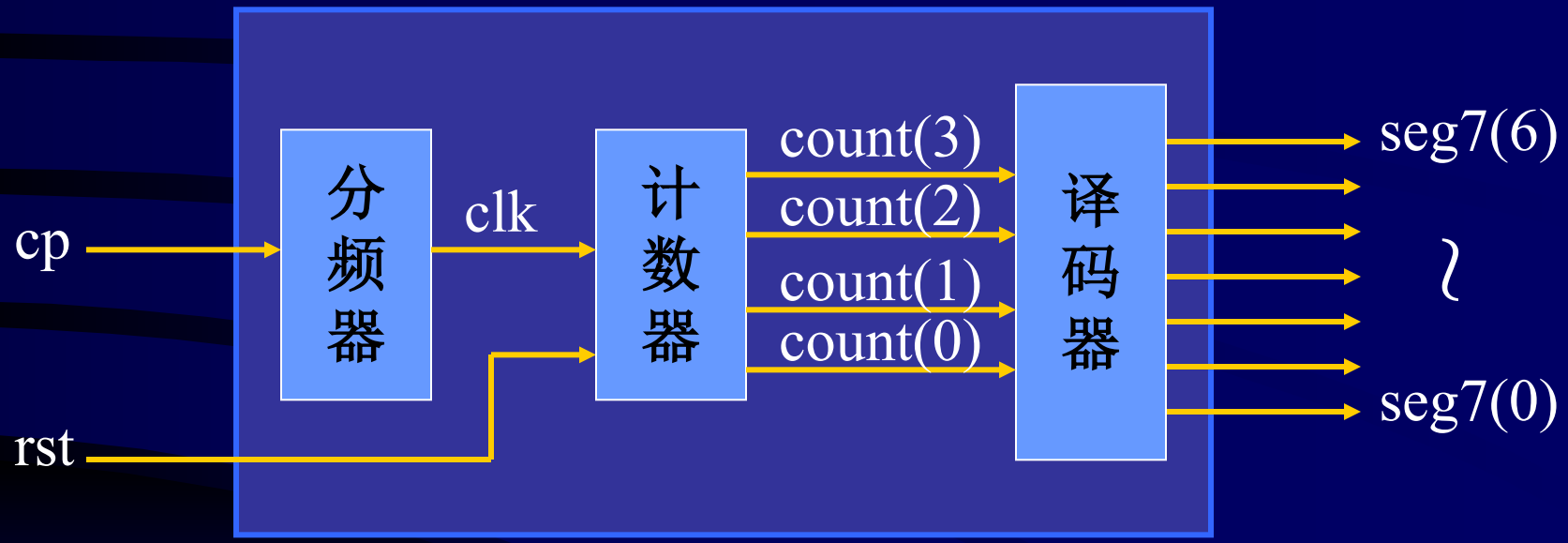
（2）系统划分，进行子系统功能描述；

  - 对各子系统和控制器进行功能描述： ASM图、MDS图和硬件描述语言。

（3）逻辑描述，完成具体设计。

整个设计过程是一个反复修改和补充的过程。

- 例：采用自顶向下模块化设计方法实现十进制加法计数器的7段数码显示。



```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
ENTITY display1 IS
    PORT ( cp, rst : IN STD_LOGIC;
           seg7 : OUT STD_LOGIC_VECTOR (6 DOWNTO 0));
END display1;
ARCHITECTURE  a OF display1 IS
  COMPONENT  fenpin
      PORT(cp: IN  STD_LOGIC;
           clk: OUT   STD_LOGIC);
  END COMPONENT;
  COMPONENT counter
      PORT(clk, rst: IN  STD_LOGIC;
           q: OUT    STD_LOGIC_VECTOR(3 DOWNTO 0));
  END COMPONENT;
  COMPONENT segment
      PORT(count: IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
           seg7: OUT   STD_LOGIC_VECTOR(6 DOWNTO 0));
  END COMPONENT;
```

顶层文件

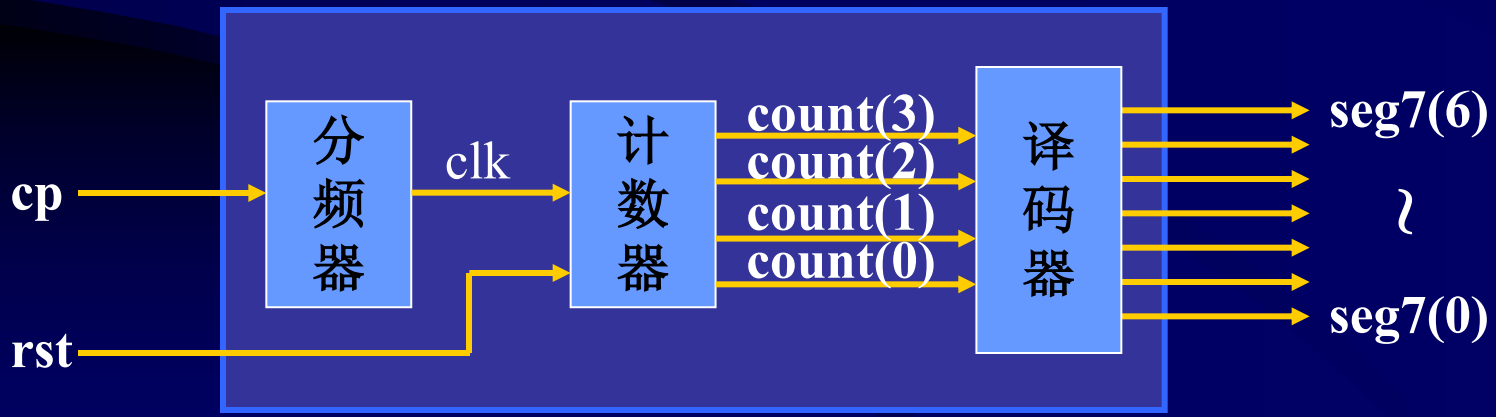
元件调用声明

接下页

接上页

```
    SIGNAL clk : STD_LOGIC ;
    SIGNAL count:STD_LOGIC_VECTOR(3 DOWNTO 0);
BEGIN
    u0: fenpin PORT MAP (cp => cp, clk => clk);
    u1: counter PORT MAP (clk => clk, rst => rst, q => count);
    u2: segment PORT MAP (count => count, seg7 => seg7);
END  a;
```

元件例化



```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
ENTITY fenpin IS
    PORT ( cp : IN   STD_LOGIC;
           clk : OUT STD_LOGIC);
END  fenpin;
ARCHITECTURE  a  OF  fenpin  IS
    SIGNAL tout : INTEGER   range  0  to  9;
BEGIN
    PROCESS(cp)
    BEGIN
        IF  (cp'EVENT and cp='1') THEN
            IF  tout = 9  THEN   tout <= 0;
            ELSE    tout <= tout + 1;
            END IF;
            IF  tout <=4  THEN    clk <= '0';
            ELSE    clk <= '1';
            END IF;
        END IF;
    END PROCESS;
END  a;
```

底层：分频器

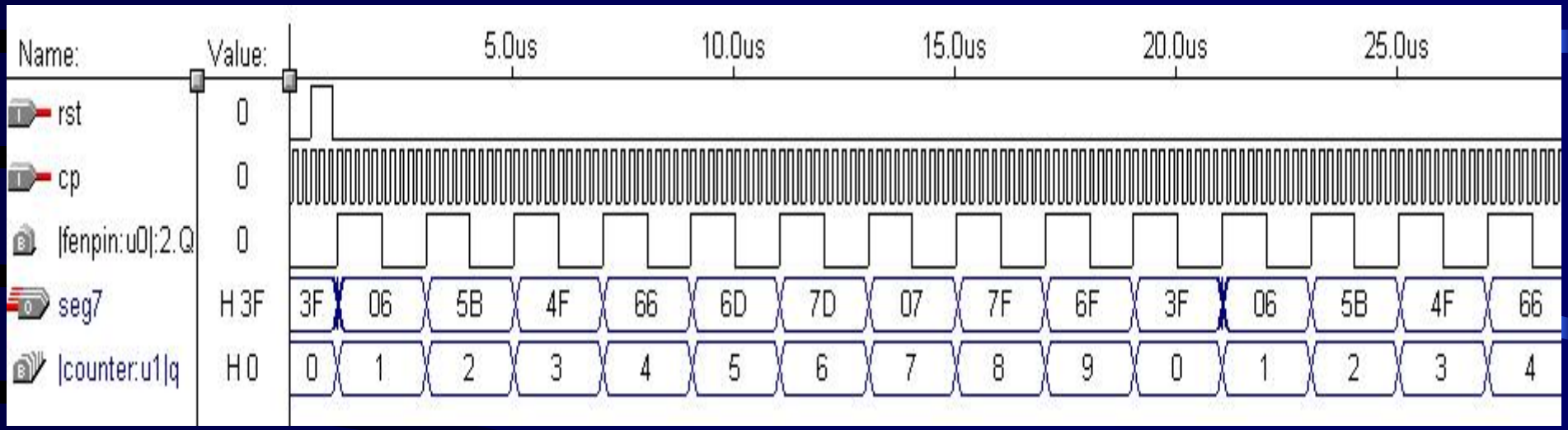
降低分频比
以便仿真

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;
ENTITY counter IS
    PORT (clk, rst : IN STD_LOGIC;
          q : OUT STD_LOGIC_VECTOR (3 DOWNTO 0));
END counter;
ARCHITECTURE  a  OF counter IS
    SIGNAL count : STD_LOGIC_VECTOR(3 DOWNTO 0);
BEGIN
    PROCESS(clk, rst)
    BEGIN
        IF rst = '1'  THEN   count <= "0000";
        ELSIF clk'EVENT AND clk = '1' THEN
            IF count >= "1001"  THEN    count <= "0000";
            ELSE     count <= count + 1;
            END IF;
        END IF;
    END PROCESS ;
    q <= count;
END   a;
```

底层：计数器

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
ENTITY segment IS
     PORT (count : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
                seg7 : OUT STD_LOGIC_VECTOR(6 DOWNTO 0));
END segment;
ARCHITECTURE a OF segment IS
BEGIN
    PROCESS (count)
    BEGIN
       CASE count IS
            WHEN "0000" =>seg7 <="0111111" ;  WHEN "0001" => seg7 <= "0000110" ;
            WHEN "0010" =>seg7 <="1011011" ;  WHEN "0011" => seg7 <= "1001111" ;
            WHEN "0100" =>seg7 <="1100110“ ;  WHEN "0101" => seg7 <= "1101101" ;
            WHEN "0110" =>seg7 <="1111101" ;  WHEN "0111" => seg7 <= "0000111" ;
            WHEN "1000" =>seg7 <="1111111“ ;  WHEN "1001" => seg7 <= "1101111" ;
            WHEN OTHERS =>  NULL ;
       END CASE ;
    END PROCESS ;
END   a;
```

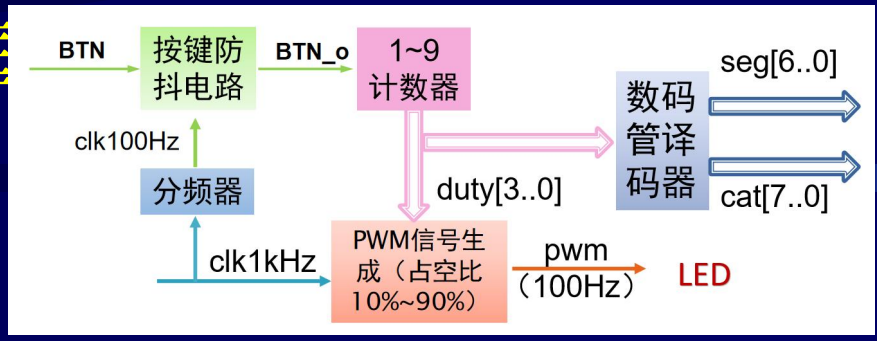
底层：译码器


Name:
Value:
5.0us
10.0us
15.0us
20.0us
25.0us
rst
0
cp
0
|fenpin:u0|:2.Q
0
seg7
H 3F
3F 06 5B 4F 66 6D 7D 07 7F 6F 3F 06 5B 4F 66
|counter:u1|q
H 0
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4

时序仿真波形

三、数字系统设计的描述方法

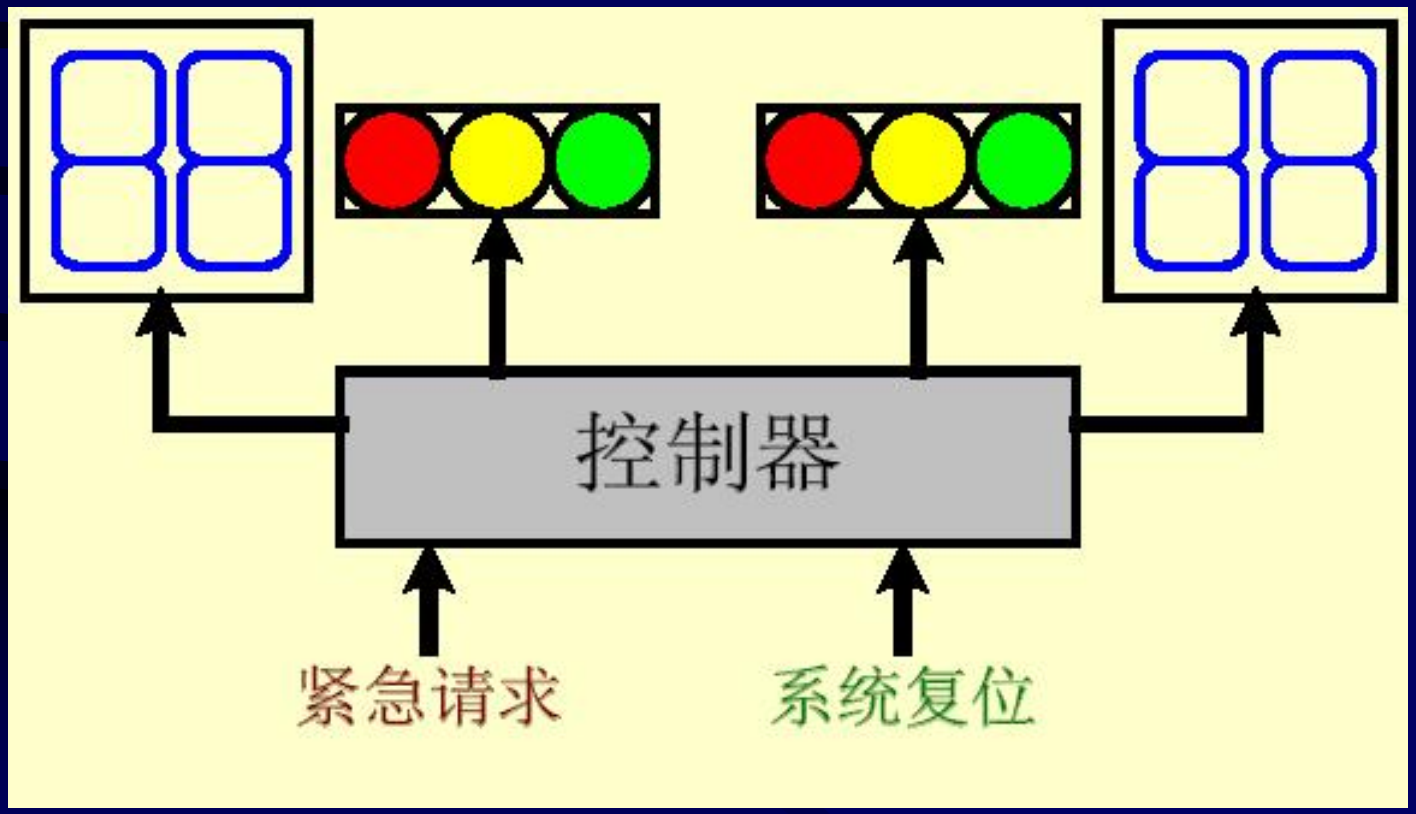
常用的描述方法：方框图、ASM图和MDS图。

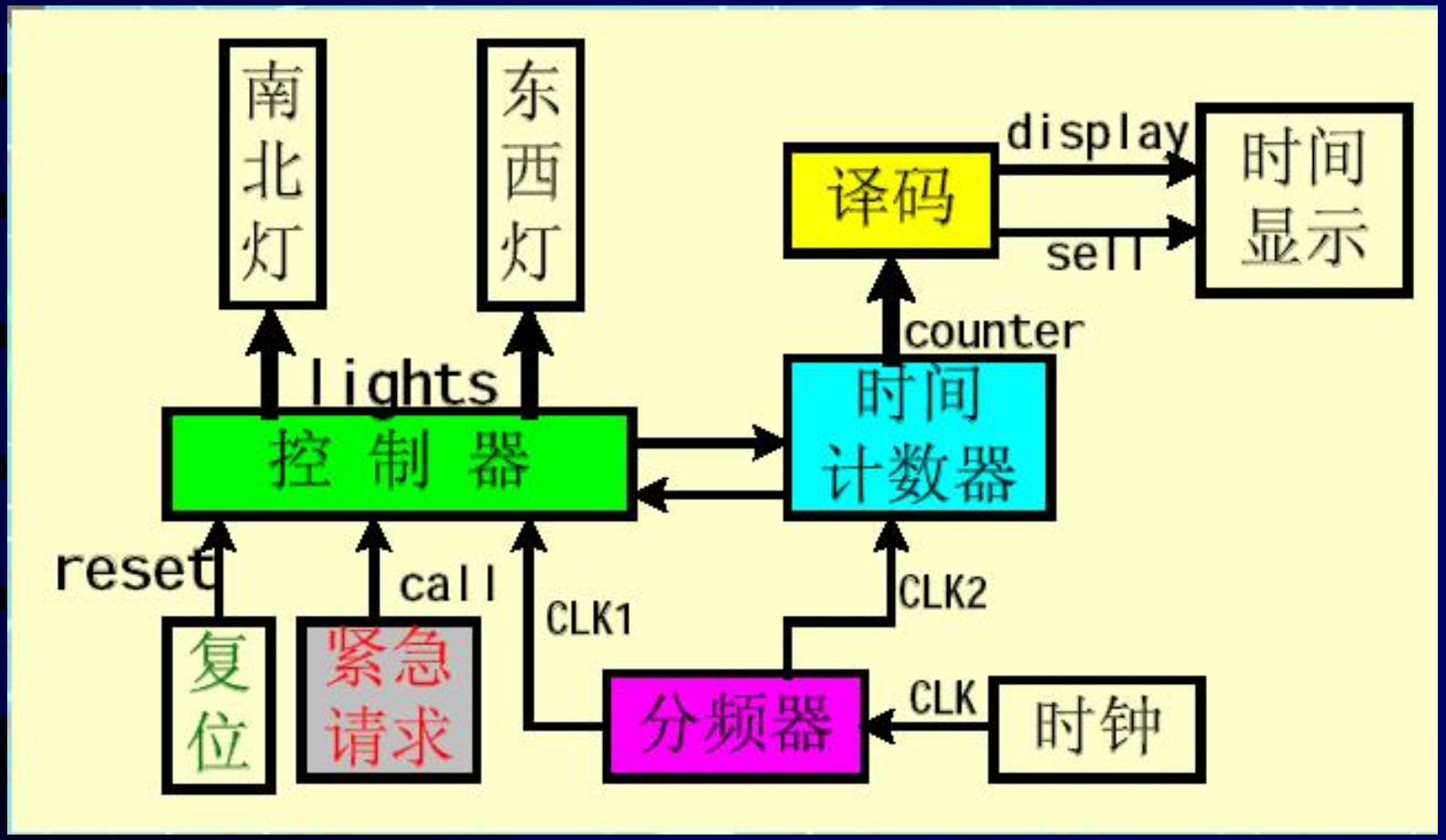


1、方框图

- ✓ 方框图详细描述数字系统的总体结构，不涉及过多的技术细节，直观易懂。
- ✓ 方框图中每一个方框定义一个信息处理、存储或传送的子系统，在方框内用文字、表达式、通用符号或图形来表示该子系统的名称或主要功能。
- ✓ 方框之间用带箭头的直线相连，表示子系统之间数据流或控制流的信息通道，箭头表示信息传送方向。

- 例：设计制作一个用于十字路口的交通灯自动控制器。





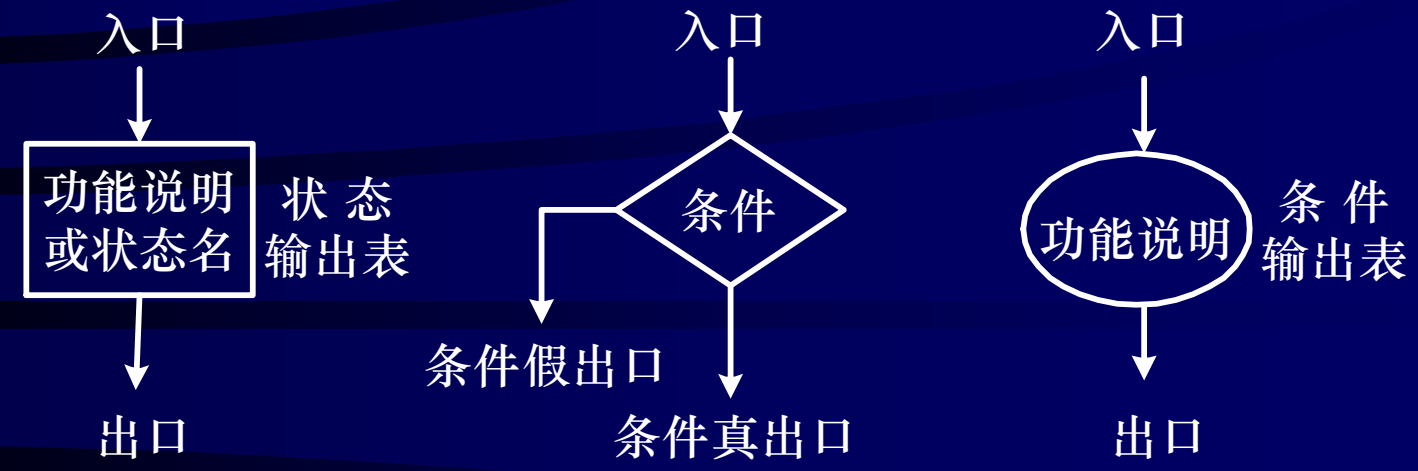
交通灯控制器的逻辑划分方框图

2、定时图（时序图、时间关系图）

- 用来定时描述系统各模块之间、模块内部各功能组件之间以及组件内部各个门电路或触发器之间输入信号、输出信号和控制信号的对应时序关系和特征（信号是电平还是脉冲，是同步信号还是异步信号等）。
- 定时图精确地定义了系统的功能。在系统调试时，借助EDA工具，建立系统的模拟仿真波形，以判定系统中可能存在的错误；或在硬件调试及运行时，可通过逻辑分析仪或示波器对系统中重要结点处的信号进行观测，以判断系统中可能存在的错误。

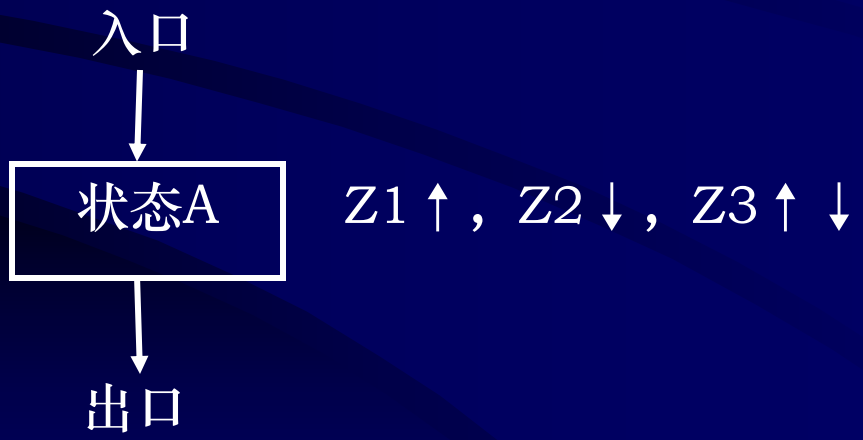
3、逻辑流程图（流程图）

- 是用特定的几何图形（如矩形、菱形、椭圆等）、指向线和简练的文字说明，来描述数字系统的基本工作过程。其描述对象是控制单元，并以系统时钟来驱动整个流程。
- 描述整个数字系统对信息的处理过程，以及控制单元所提供的控制步骤，以便于设计者发现和改进信息处理过程中的错误和不足，同时又是后续电路设计的依据。

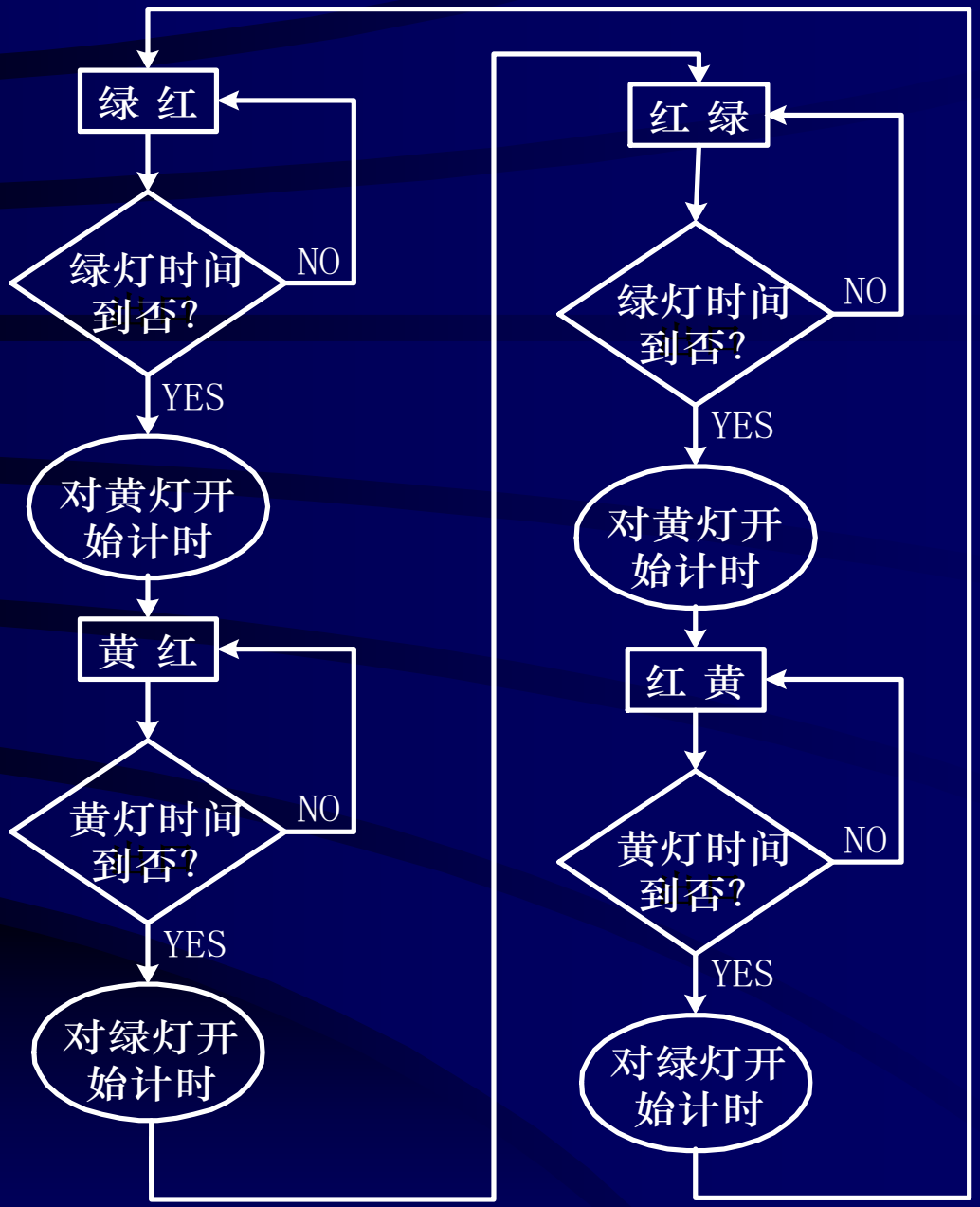


（A） 状态框　　（B）条件判别框　　（C）条件输出框

逻辑流程图基本符号



状态输出表



交通灯控制器的逻辑流程图

4、算法状态机图（ASM图）

- ASM图是描述时钟控制流程的图形方式；
- 符号及符号的意义与逻辑流程图相似；
- 与逻辑流程图的区别不大，可以说ASM图是详细的逻辑流程图；
- 可以由逻辑流程图导出ASM图。

5、MDS图

- MDS图（助记状态图）类似于状态转换图，可以由流程图转换而来，是设计数字系统控制器（状态机）的一种简洁方法。
- MDS图是用一个圆圈表示一个状态，状态名标注在圆圈内，圆圈外的符号或逻辑表达式表示输出，用定向线表示状态转换方向，定向线旁的符号或逻辑表达式表示转换条件。
- 控制器的实现：将MDS图转化成硬件描述语言，送入计算机，由EDA工具自动完成控制器的设计；也可采用人工进行设计。

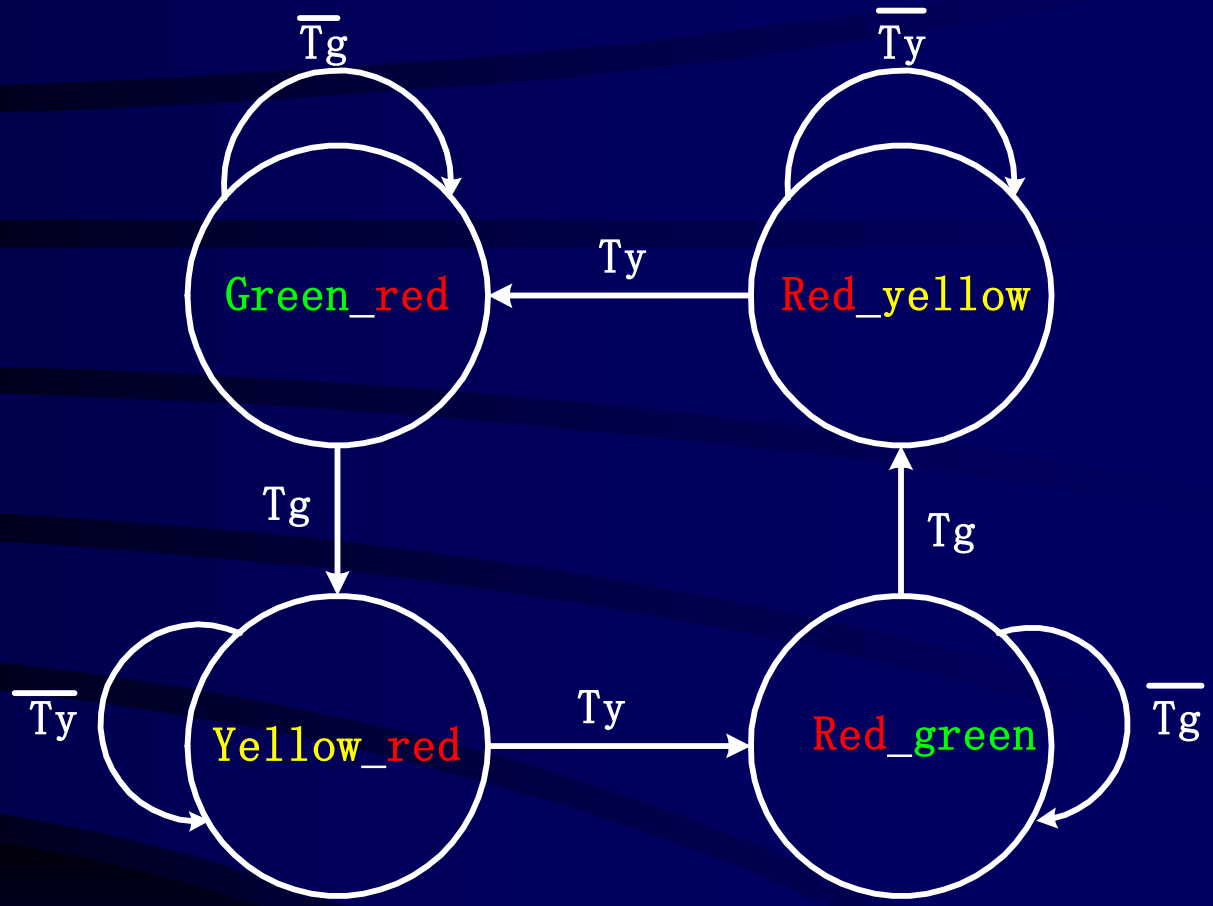


图7 交通灯控制器的MDS图

四、有限状态机的设计

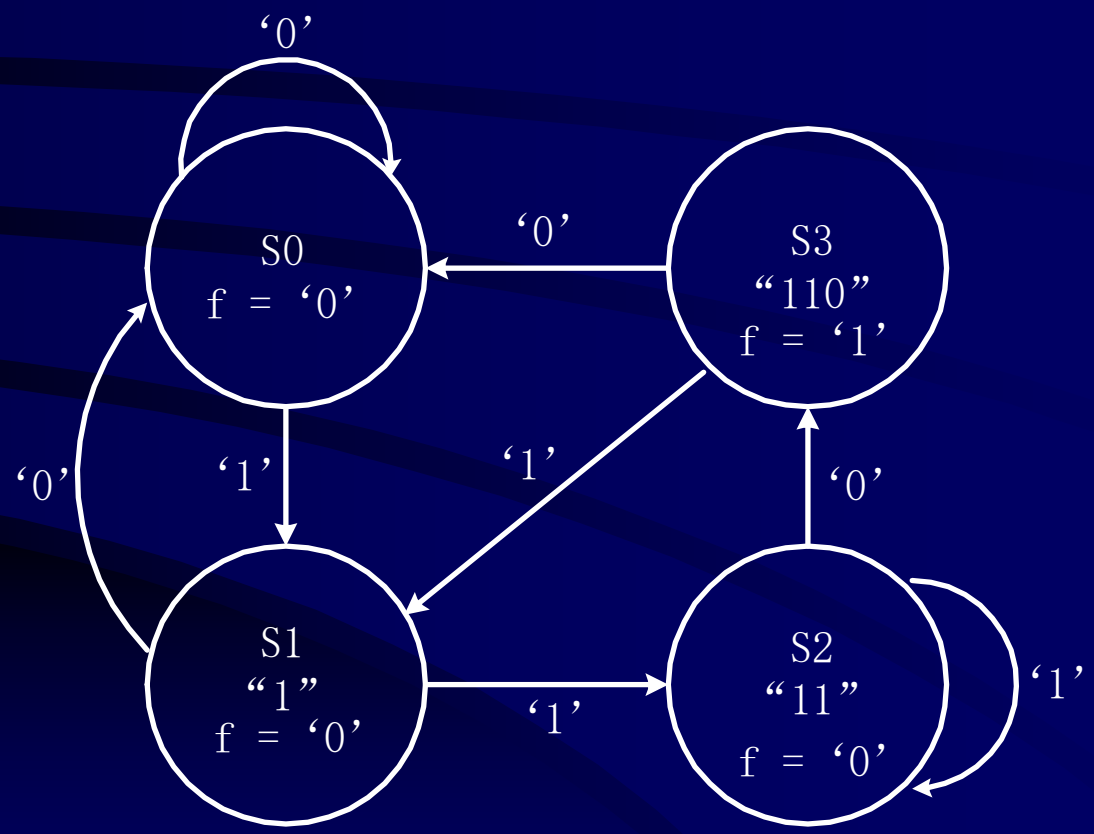
- 有限状态机是广义的时序电路，主要功能是用来实现数字系统中的控制单元。
- 有限状态机的两种类型
  - Moore型有限状态机：输出信号仅与当前状态有关。
  - Mealy型有限状态机：输出信号不仅与当前状态有关，还与所有的输入信号有关。

两种有限状态机在VHDL描述上的差别较小。

- 有限状态机的描述规则
  - 至少包括一个状态信号，用来指定有限状态机的状态；
  - 状态转移指定和输出指定，对应于控制单元中与每个控制步有关的转移条件；
  - 时钟信号，用来进行同步；
  - 同步或异步复位信号，不是必需的，但对于实际应用的有限状态机，复位信号是不可缺少的。
  - 经常使用的描述方式有三种：三进程描述、双进程描述和单进程描述。

描述方式		进程描述的功能	进程数
三进程描述方式		进程1：描述次态逻辑 进程2：描述状态寄存器 进程3：描述输出逻辑	3
双进程描述方式	形式1	进程1：描述次态逻辑、输出逻辑 进程2：描述状态寄存器	2
	形式2	进程1：描述次态逻辑、状态寄存器 进程2：描述输出逻辑	2
	形式3	进程1：描述状态寄存器、输出逻辑 进程2：描述次态逻辑	2
单进程描述方式		进程1：描述次态逻辑、状态寄存器和输出逻辑	1

例： Moore型有限状态机的VHDL描述。状态转移图如下图所示，该序列信号检测器连续接收到一组“110”后输出为‘1’，其它情况下输出为‘0’。



三进程描述方式

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
ENTITY   threepro  IS
      PORT( clk : IN  STD_LOGIC;
            d_in : IN  STD_LOGIC;
            f : OUT  STD_LOGIC );
END  threepro;
ARCHITECTURE  a  OF threepro  IS
    TYPE  state  IS  (S0, S1, S2, S3);
    SIGNAL  presentstate, nextstate : state;
BEGIN
    p1 : PROCESS (clk)
    BEGIN
        IF reset = '1' THEN
            presentstate <= s0;
        ELSIF  clk'event  AND  clk = '1'  THEN
            presentstate <= nextstate ;
        END IF;
    END  PROCESS;
```

自定义可枚举类型

TYPE 数据类型名 IS 数据类型定义 OF 基本数据类型 ；

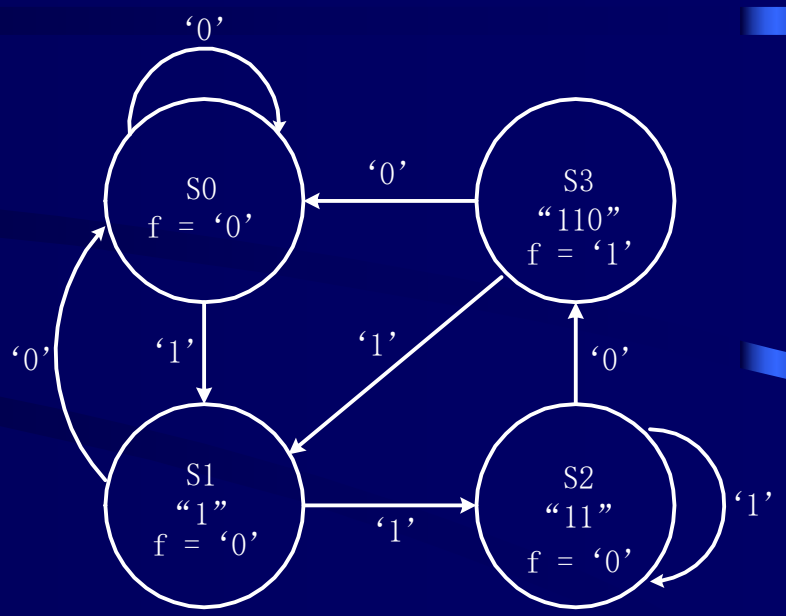
TYPE 数据类型名 IS 数据类型定义 ；

描述状态寄存器

接下页

描述次态逻辑

```
p2 : PROCESS (d_in , presentstate)
BEGIN
    CASE  presentstate  IS
        WHEN S0 =>  IF d_in = '1'  THEN
                        nextstate <= S1;
                    ELSE
                        nextstate <= S0;
                    END IF;
        WHEN S1 =>  IF d_in = '1'  THEN
                        nextstate <= S2;
                    ELSE
                        nextstate <= S0;
                    END IF;
        WHEN S2 =>  IF d_in = '0'  THEN
                        nextstate <= S3;
                    ELSE
                        nextstate <= S2;
                    END IF;
```

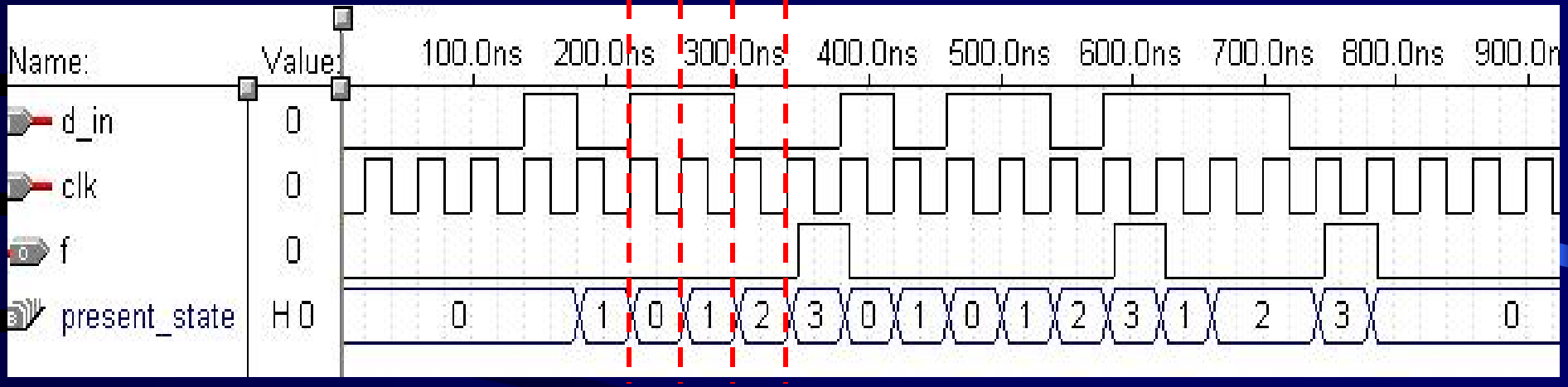


接下页

接上页

```
                WHEN S3  =>  IF d_in = '0'   THEN
                                    nextstate <= S0;
                             ELSE
                                    nextstate <= S1;
                             END IF;
        END  CASE;
    END  PROCESS;
    p3 : PROCESS ( presentstate )
    BEGIN
        CASE  presentstate  IS
            WHEN s0 => f <= '0';
            WHEN s1 => f <= '0';
            WHEN s2 => f <= '0';
            WHEN s3 => f <= '1';
      END CASE;
    END PROCESS ;
END a;
```

描述输出逻辑


Name:
Value:
100.0ns
200.0ns
300.0ns
400.0ns
500.0ns
600.0ns
700.0ns
800.0ns
d_in
0
clk
0
f
0
present_state
H 0
0
1
0
1
2
3
0
1
0
1
2
3
1
2
3
0

状态机的工作时序

双进程描述方式

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
ENTITY   twopro  IS
      PORT( clk : IN  STD_LOGIC;
            d_in : IN  STD_LOGIC;
            f : OUT  STD_LOGIC );
END  twopro;
ARCHITECTURE  a  OF twopro  IS
    TYPE  state  IS  (S0, S1, S2, S3);
    SIGNAL  presentstate, nextstate : state;
BEGIN
    p1 : PROCESS (clk)
    BEGIN
        IF  clk'event  AND  clk = '1'  THEN
            presentstate <= nextstate ;
        END IF;
    END  PROCESS;
```

描述状态寄存器

接下页

接上页

描述次态逻辑及输出逻辑

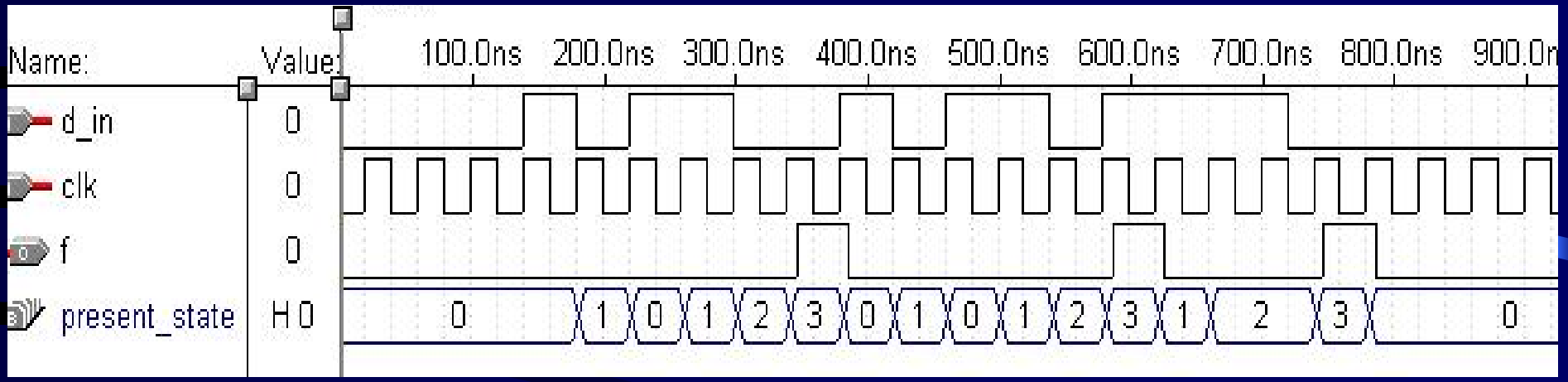
```
p2 : PROCESS (d_in , presentstate)
BEGIN
      CASE  presentstate  IS
            WHEN S0 =>  IF d_in = ‘1'  THEN
                              nextstate <= S1;
                        ELSE
                              nextstate <= S0;
                        END IF;
                              f <= '0';
             WHEN S1 =>  IF d_in = '1'  THEN
                              nextstate <= S2;
                         ELSE
                              nextstate <= S0;
                        END IF;
                              f <= ‘0';
```

接下页

接上页

```
            WHEN S2 =>   IF d_in = '0'  THEN
                              nextstate <= S3;
                         ELSE
                              nextstate <= S2;
                         END IF;
                              f <= '0';
            WHEN S3 =>   IF d_in = '0'   THEN
                               nextstate <= S0;
                         ELSE
                               nextstate <= S1;
                         END IF;
                               f <= '1';

        END  CASE;
    END  PROCESS;
END a;
```


Name:
Value:
100.0ns
200.0ns
300.0ns
400.0ns
500.0ns
600.0ns
700.0ns
800.0ns
d_in
0
clk
0
f
0
present_state
H 0
0
1
0
1
2
3
0
1
0
1
2
3
1
2
3
0

状态机的工作时序

单进程描述方式

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
ENTITY   onepro  IS
      PORT( clk : IN  STD_LOGIC;
            d_in : IN  STD_LOGIC;
            f : OUT  STD_LOGIC );
END  onepro;
ARCHITECTURE  a  OF onepro  IS
     TYPE  state  IS  (S0, S1, S2, S3);
     SIGNAL  nstate : state;
BEGIN
     PROCESS (clk)
     BEGIN
          IF  clk'event  AND  clk = '1'  THEN
               CASE  nstate  IS
                    WHEN S0 => IF ( d_in= ‘1‘ ) THEN
                                    nstate <=S1;
```

接下页

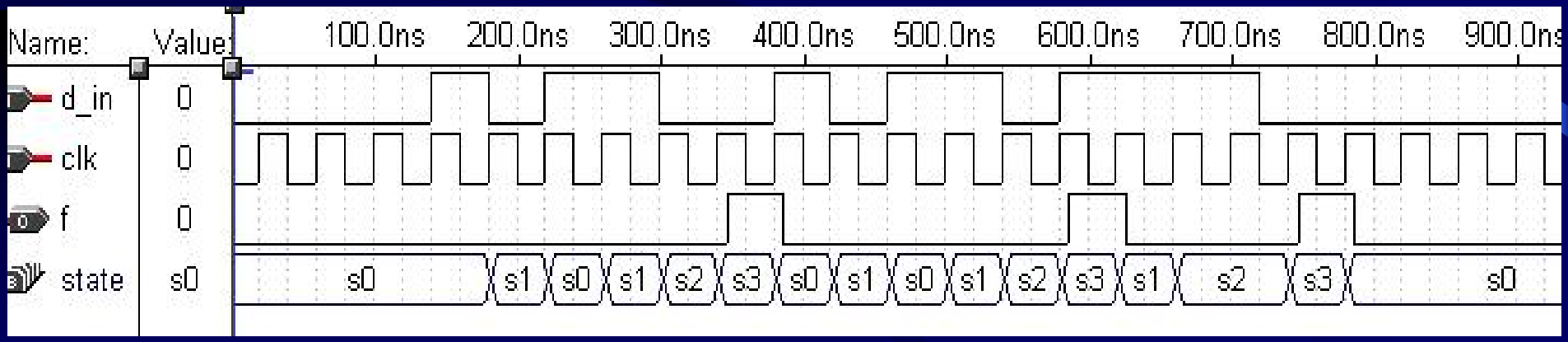
接上页

```
                ELSE
                    nstate <= S0;
                END IF;
                    f <= '0';
WHEN S1 =>  IF ( d_in = '1‘ ) THEN
                    nstate <= S2;
                ELSE
                    nstate <= S0;
                END IF;
                    f <= ‘0';
WHEN S2 =>  IF ( d_in = ‘0' ) THEN
                    nstate <= S3;
                ELSE
                    nstate <= S2;
                END IF;
                    f <= '0';
```

接下页

接上页

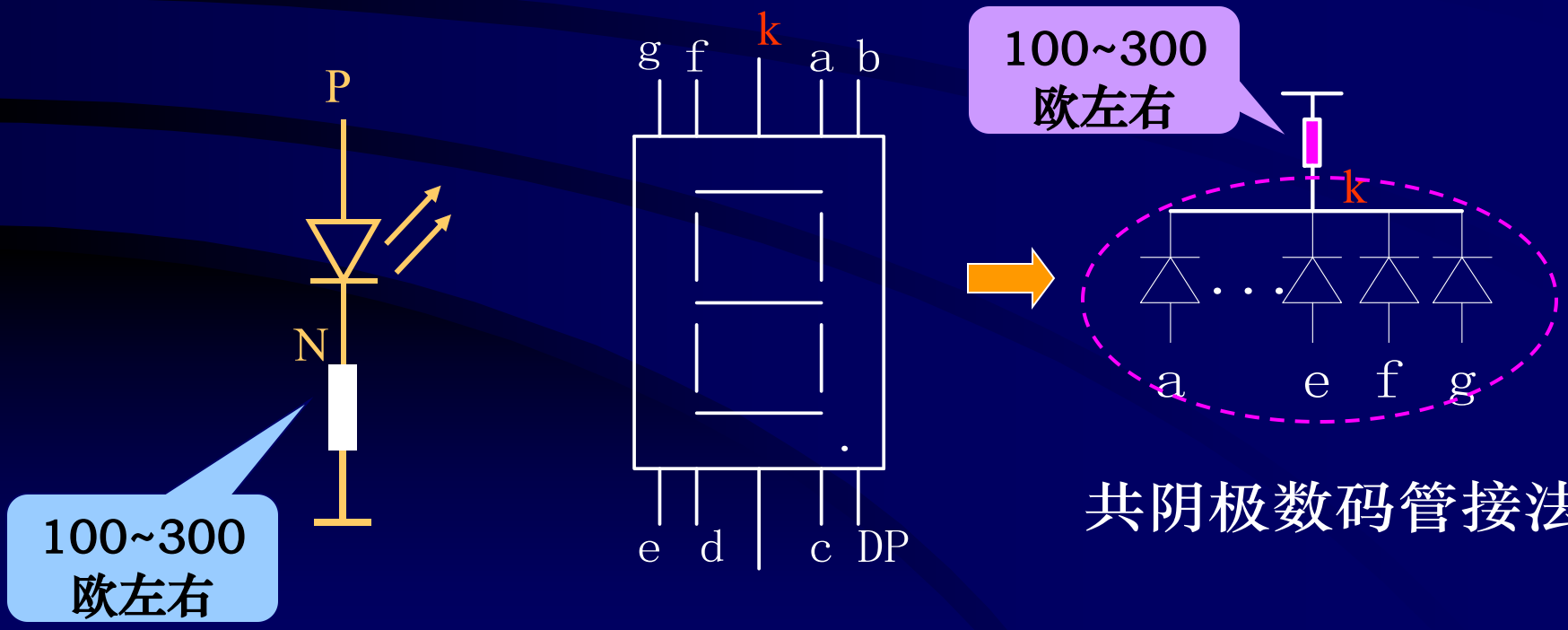
```
            WHEN S3 => IF ( d_in = '0' ) THEN
                                nstate <= S0;
                       ELSE
                                nstate <= S1;
                       END IF;
                                f <= '1';
          END  CASE;
      END IF;
  END  PROCESS;
END a;
```



五、数字系统设计时注意的问题

（一）开机复位：初值只具有仿真意义，不能实现对系统的复位，复位必须用按键来实现。

（二）限流保护：数码管、发光二极管、点阵、按键等要加限流保护电阻。



共阴极数码管接法

五、数字系统设计时注意的问题

（三）器件资源的优化：使用EPM1270时注意节省器件资源。

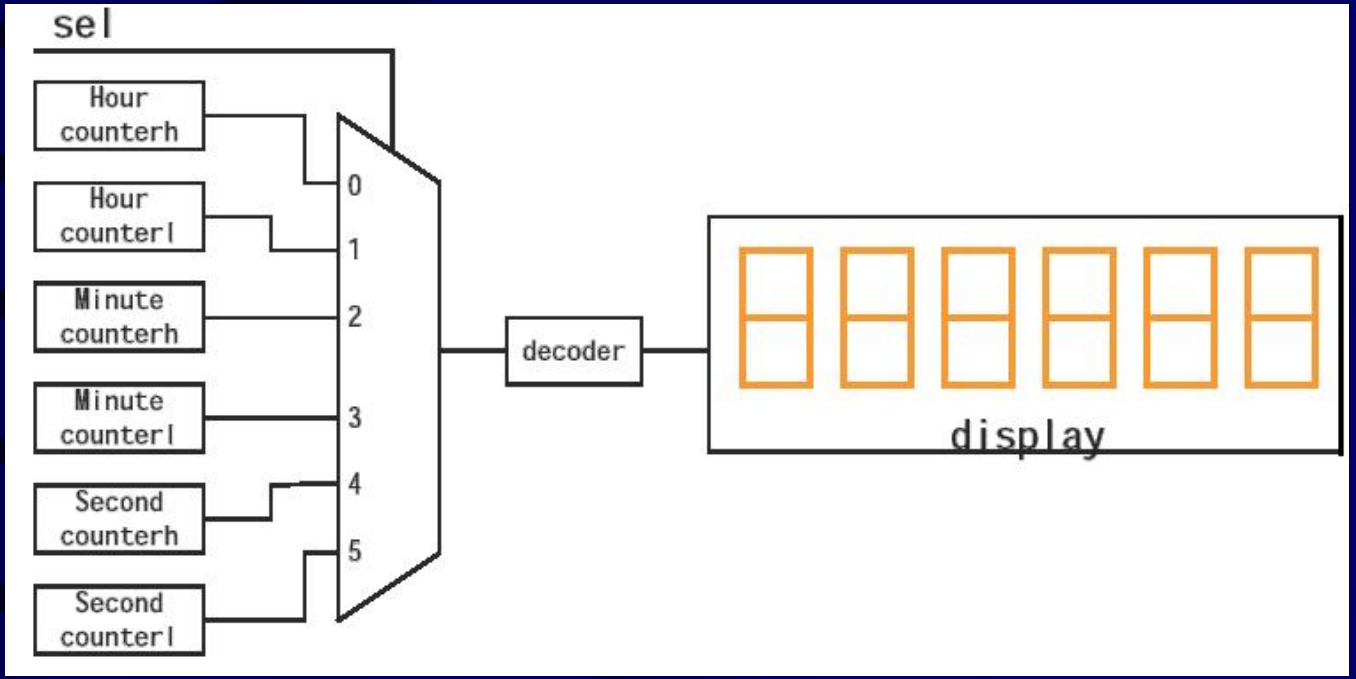
1、资源共享：针对耗费逻辑资源比较多的模块，通过选择、复用的方式共享该模块，以减少资源使用。

2、串行化：把耗用资源较多、并行执行的逻辑块分割开来，提取相同的逻辑模块，在时间上复用该逻辑模块。

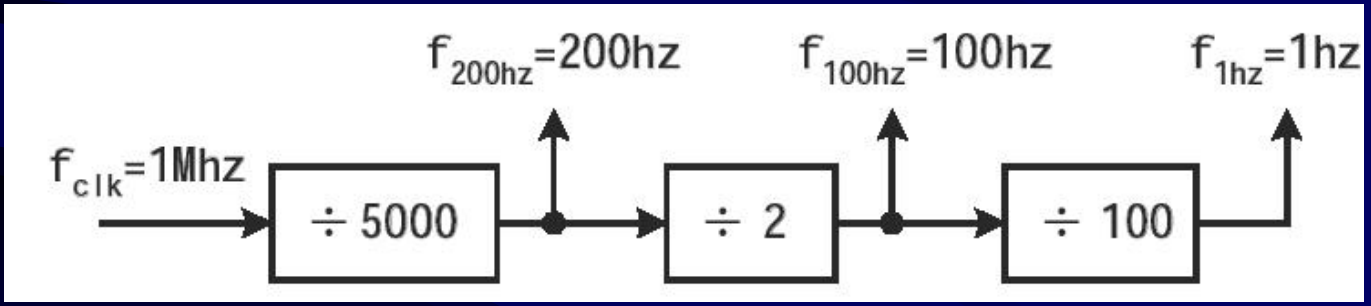
3、正确使用VHDL描述：不完整IF语句的综合结果会导致时序电路，从而导致资源浪费。

4、软件优化

例1：


sel
Hour counterh
Hour counterl
Minute counterh
Minute counterl
Second counterh
Second counterl
0
1
2
3
4
5
decoder
display

例2：


f_{clk}=1Mhz
÷ 5000
f_{200hz}=200hz
÷ 2
f_{100hz}=100hz
÷ 100
f_{1hz}=1hz

高速系统慎用！

例3：

```
IF (count3="00") THEN
      ctemp<="0001";
ELSIF(count3="01") THEN
      ctemp<="0010";
ELSIF (count3="10") THEN
      ctemp<="0100";
ELSE  ctemp<="1000";
END IF;
```

组合电路

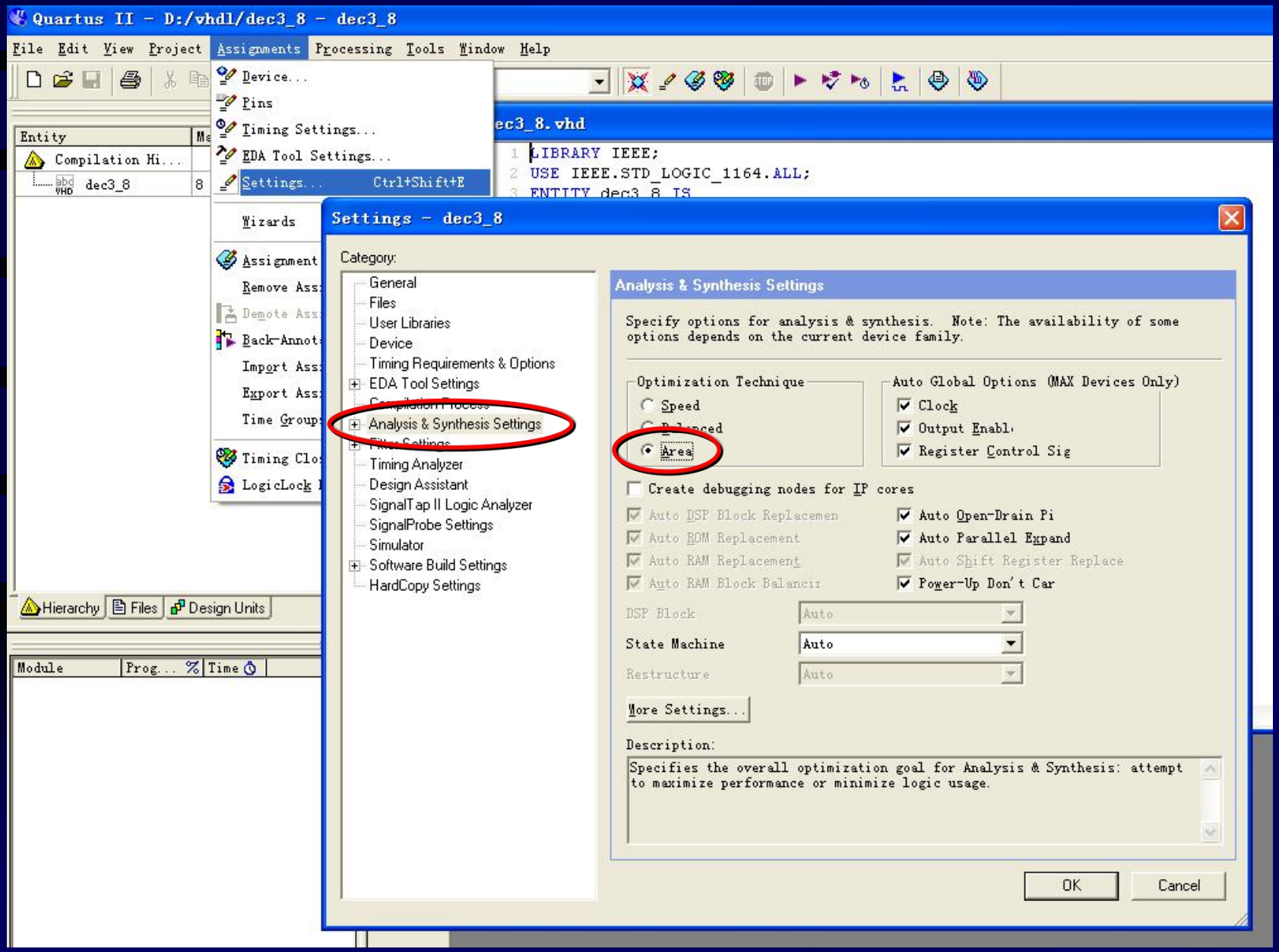
```
IF (count3="00")  THEN
      ctemp<="0001";
ELSIF (count3="01") THEN
      ctemp<="0010";
ELSIF (count3="10") THEN
     ctemp<="0100";
END IF;
```

导致产生4位锁存器

例3：

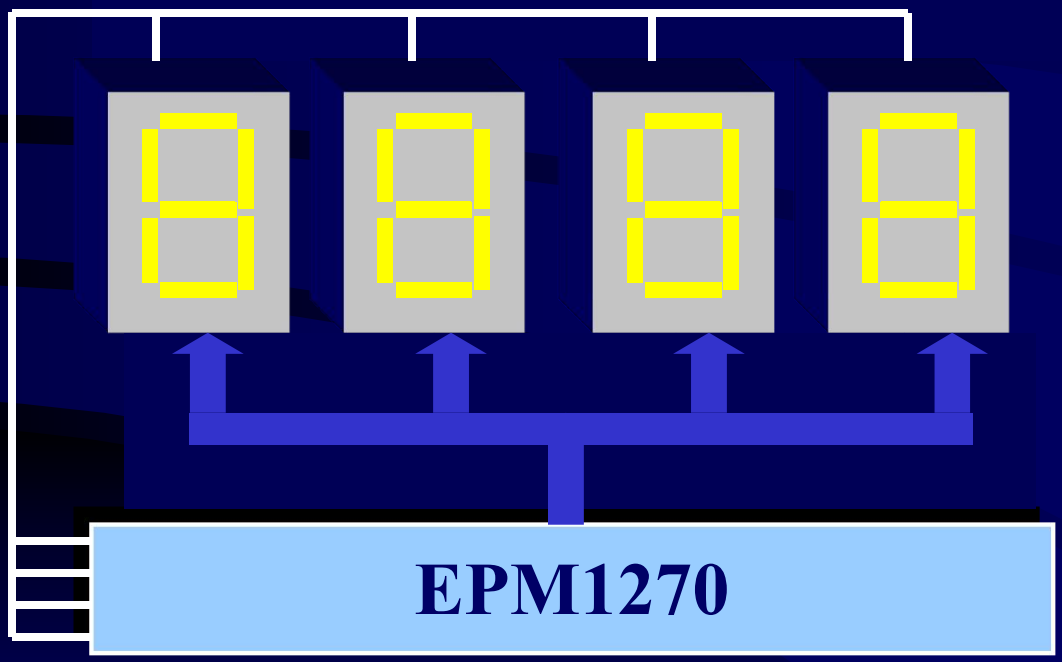
- ✓ 避免使用长的if-then-else语句，采用case语句来代替（防止出现较大的优先编码器，且提高了代码的可读性）。
- ✓ 定义整数类型的信号和变量时一定要用RANGE子句指定范围。

例4：QuartusII的优化选项



五、数字系统设计时注意的问题

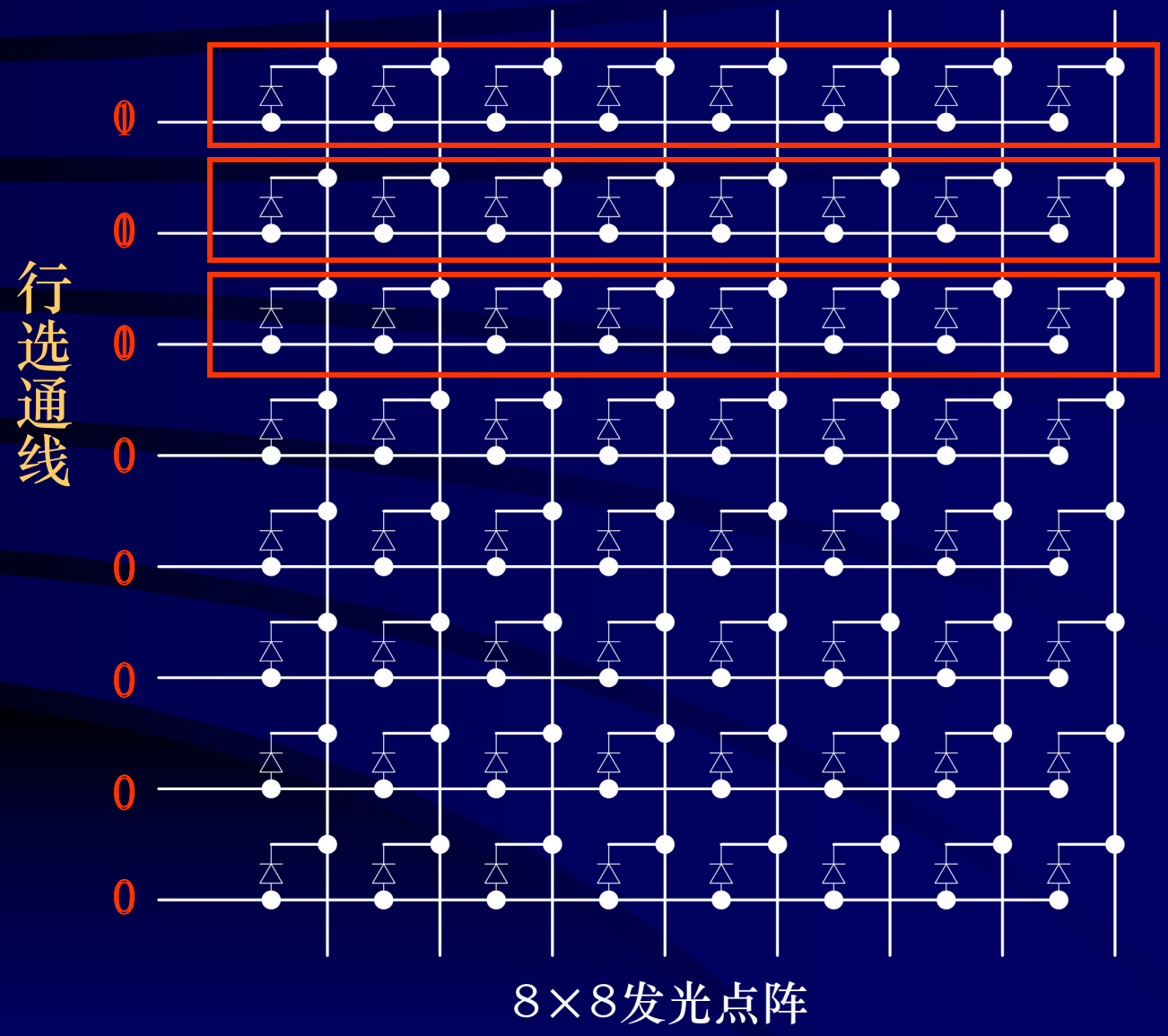
（四）显示器件的动态扫描：PLD器件直接驱动多个数码管、发光点阵使用时要采用动态扫描方式。

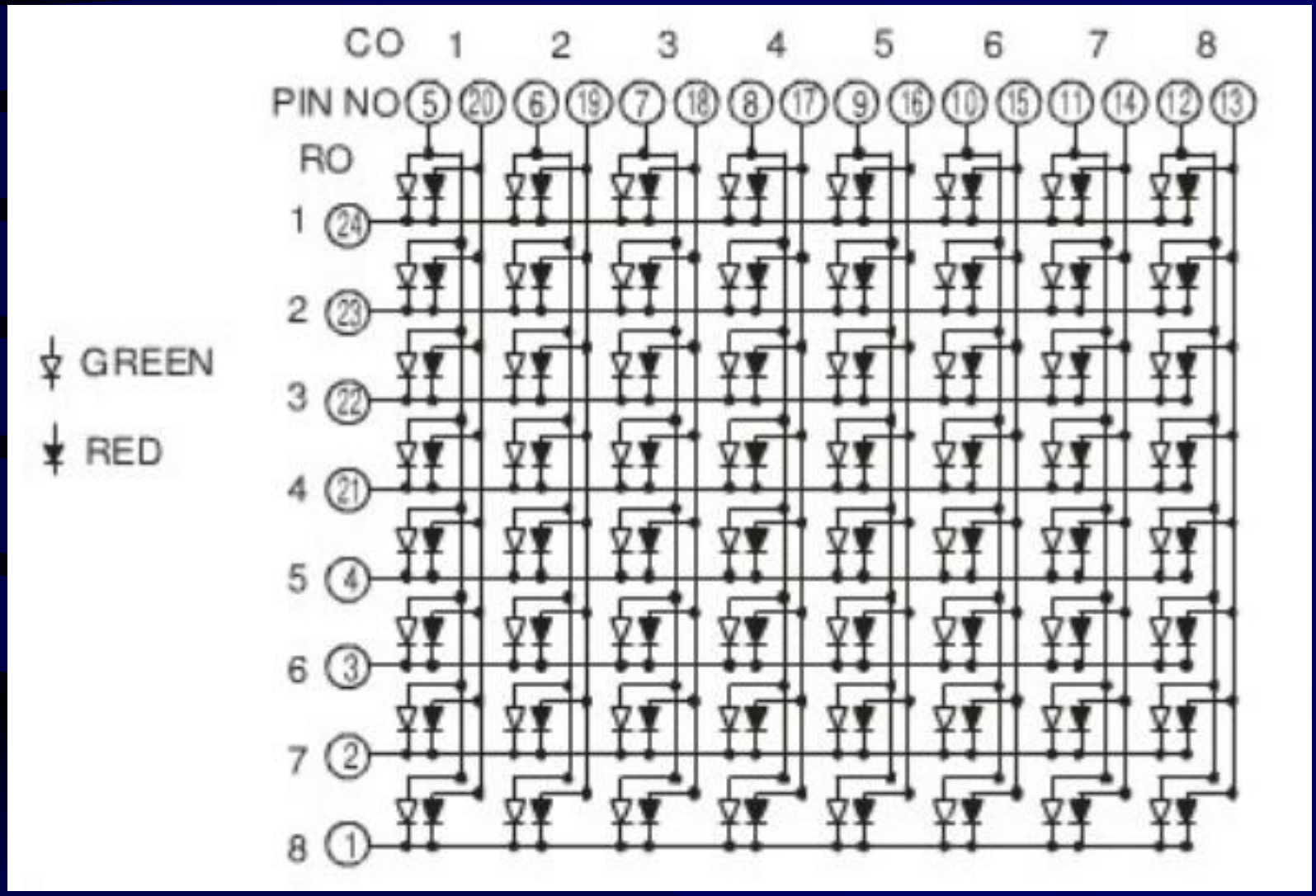


7×4=28

7+4=11

共用译码电路　减少耗电　减少对器件引脚的占用

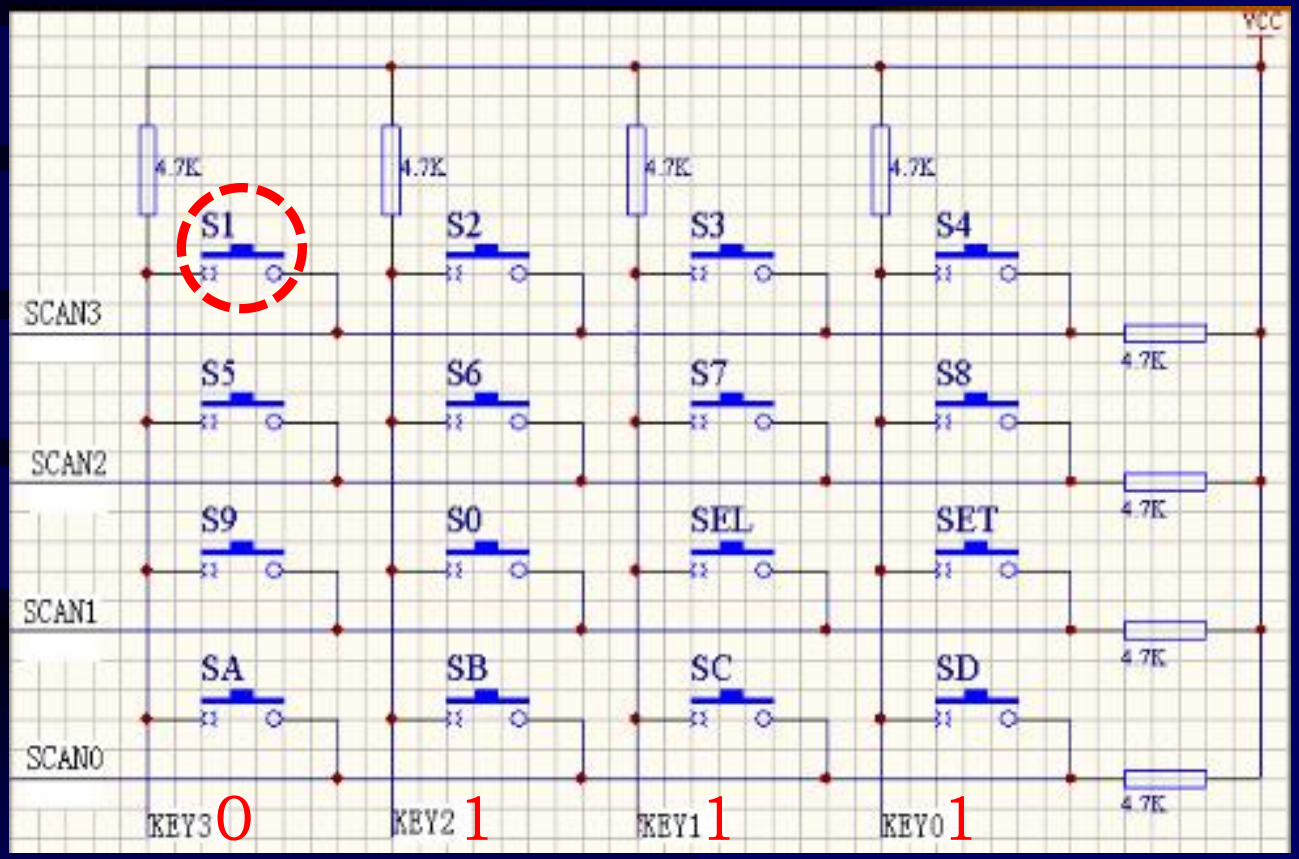

列选通线
行选通线
0
0
0
0
0
0
0
0
8×8发光点阵



采用行扫描方式进行驱动

五、数字系统设计时注意的问题

（五）4×4矩阵键盘的扫描：减少I/O口资源的占用。



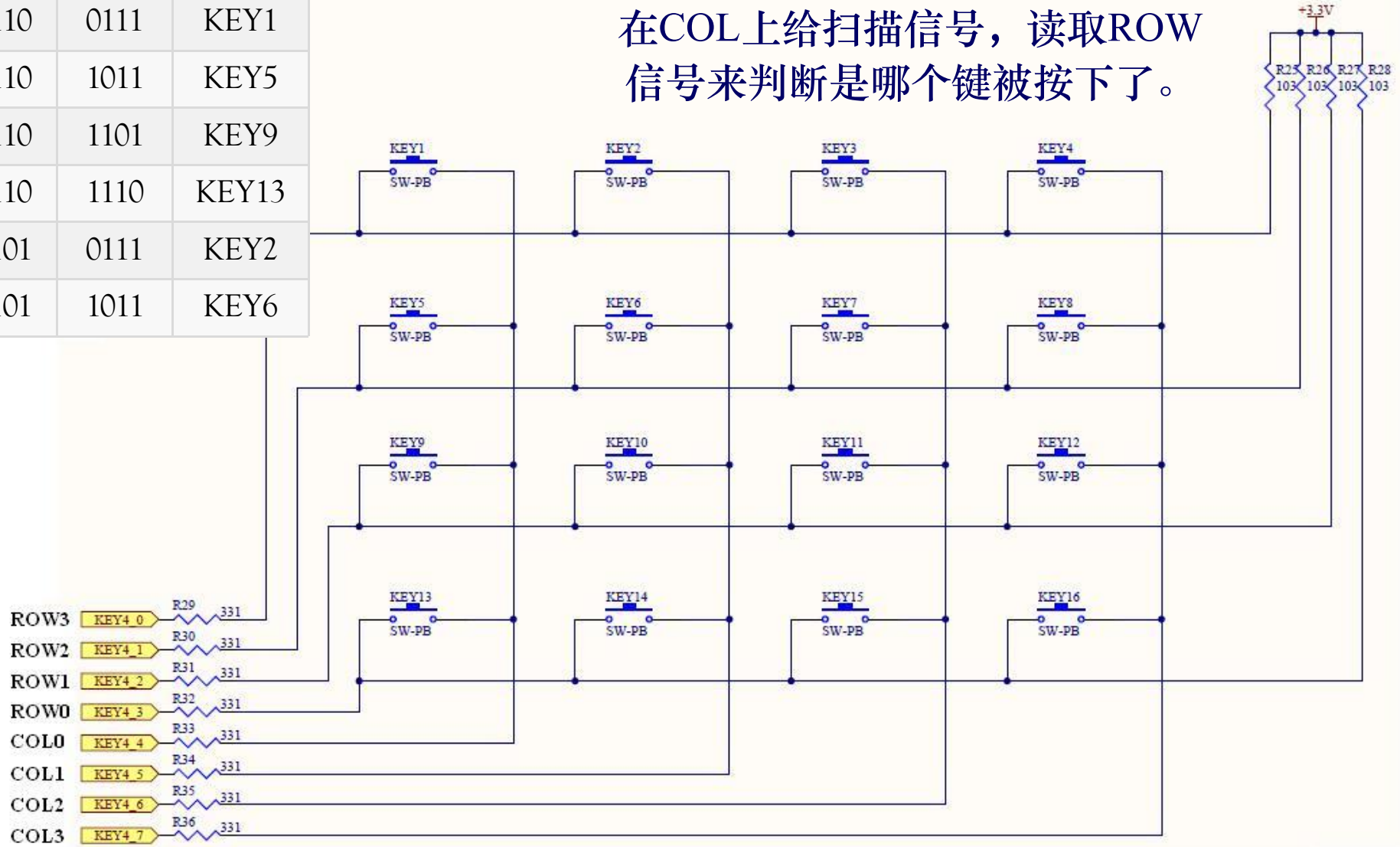
0 1 1 1

KEY3 0 KEY2 1 KEY1 1 KEY0 1

按键要注意**防抖处理**

COL	ROW	按键
1110	0111	KEY1
1110	1011	KEY5
1110	1101	KEY9
1110	1110	KEY13
1101	0111	KEY2
1101	1011	KEY6

在COL上给扫描信号，读取ROW信号来判断是哪个键被按下了。



五、数字系统设计时注意的问题

（六）LCM12864图形点阵液晶模块



LCM12864由128*64个液晶显示点组成一个128列*64行的阵列。每个显示点对应一位二进制数，1表示亮，0表示灭。其不仅可以显示数字，还可以显示汉字和图形。

LCM12864引脚说明

引脚号	符　号	电　平	功　能
1	DB_0	H / L	数据端口
2	DB_1	H / L	
3	DB_2	H / L	
4	DB_3	H / L	
5	DB_4	H / L	
6	DB_5	H / L	
7	DB_6	H / L	
8	DB_7	H / L	
9	VDD	5.0V	模块逻辑电源输入端
10	VSS	0V	逻辑电源地
11	LEDA	5.0V	LED 背光电源正端
12	/CS	L	芯片选通断，低电平有效
13	/RES	L	复位信号，低电平有效
14	A0	H/L	命令数据选通端，H：数据，L:命令
15	/WR	L	80 时序作为写信号，68 时序作为读/写信好
16	/RD	L	80 时序作为读信号，68 时序作为使能信号

五、数字系统设计时注意的问题

（七）多重驱动的处理

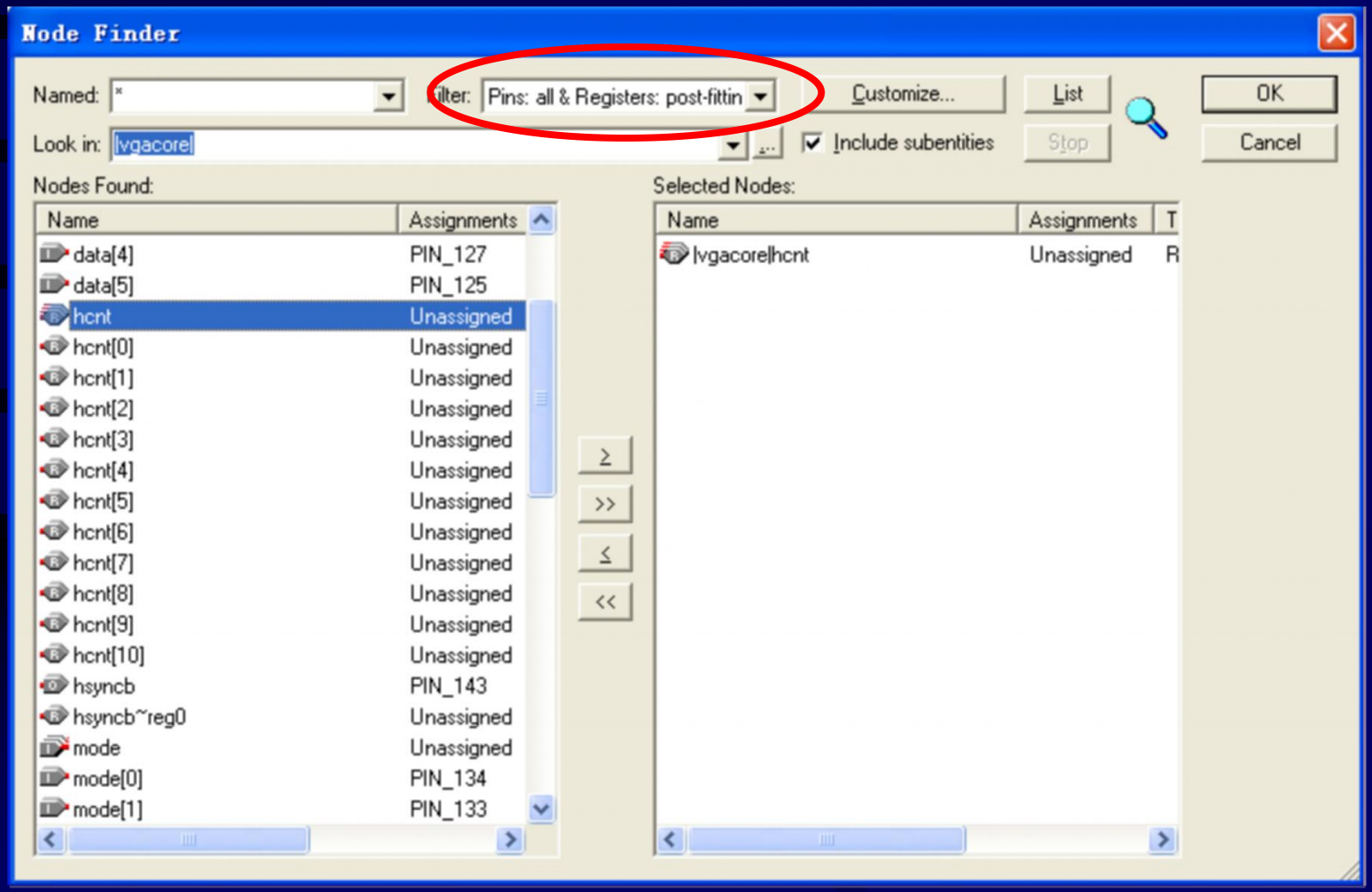
在两个及以上的进程中对同一个信号进行写操作时，编译器会提示Multi Drive错误。要从硬件角度去理解不能多重驱动的原因。

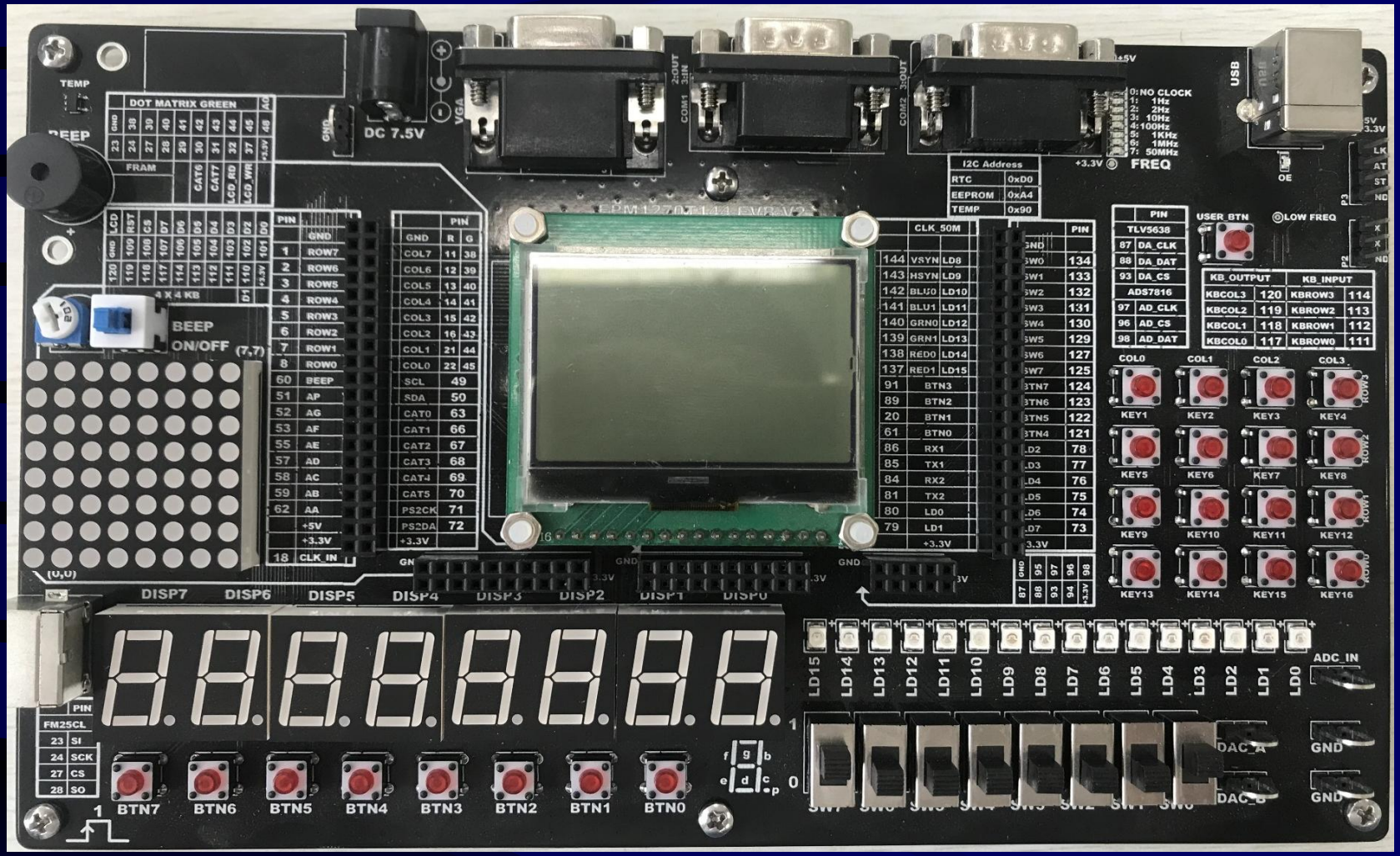
要解决多重驱动，需要至少两个信号在进程间进行通信。通过读取对方进程里面信号的值得到对方的状态，再根据对方不同状态对自己内部的信号进行操作。

```
p1: process(clk)
      if (clk'event and clk='1') then
        if (count = 5) then
          count <= 0;
        else
          count <= count + 1;
        end if;
      end if;
end process;
p2: process(count )
      if (count = 3) then
          count <=4;
      end if;
end process;
```

五、数字系统设计时注意的问题

（八）仿真：加入中间信号，便于查看电路状态变化





实验板输入时钟频率可通过右上角的USER_BTN切换选择。

蜂鸣器如果声音小，可顺时针调节点阵左上角的102电位器，注意别太用力。

温度传感器在蜂鸣器的左上角。

综合实验

一、综合实验题目

（一）个人信息轻量级保护系统的设计与实现

（二）简易雷达报警系统的设计与实现

（三）双人“五行轮转”游戏机的设计与实现

（四）简易拼图游戏的设计与实现

（五）空调遥控器的设计与实现

二、选题要求

1、题目内容和要求见教学云平台和云课堂。

2、综合实验每个题目选择人数不能超过7人。

3、根据自身的能力选择难度适当的题目，酌情挑战高难度的题目。

4、可以自拟题目，但内容及难度应与给定题目相当，并且必须经教师批准。

三、开放实验

1、微信小程序搜索“Easy云课堂”进行预约。

2、至少预约并按时参加2次课内开放实验。

3、预约时间：实验开始前7天到实验开始前30分钟。

4、签到时间：实验开始前10分钟至实验开始后30分钟。

5、预约后在规定时间内到预约的实验室，实验室的系统中将显示所有预约学生的信息，签到时系统会自动按顺序分配实验台，请务必按照分配的组号就座。

6、实验板只能在实验室内使用，不要带出实验室。

四、器材发放

1、借给每班3块实验板+下载线（登记资产编号），第8周验收时由学委收回归还，请保管好实验板。

2、保管好舵机配套的螺丝，没有富余。

3、蓝牙模块可在开发选做实验时登记领取。

4、题目1、2、5领取的材料验收答辩后要归还。

1. 舵机使用时先加电控制舵机回到0°的位置，然后用舵机配件里最小的螺丝将双向扇叶固定到舵机的轴上；
2. 用舵机配件里的自攻螺丝（尖头）将超声波模块支架固定到双向扇叶上，由于自攻螺丝较长，可以用超声波支架配件里的螺母做隔离（如图所示）；



五、报告提交要求

1、提交含实验报告、工作视频、完整project（包括源程序、仿真波形及相关文件）的压缩文件。

2、压缩文件名命名：题目-姓名。

3、实验完成后的一周之内在云课堂和教学云平台提交压缩文件。

